

MLOps Training 2026/2027

Task 1 — Get the Data Into a Database

Project: **Olist MLOps**
Database: PostgreSQL
Database Client: pgAdmin 4
Database Name: olist
Task: 01 — Database Setup and Validation
Date: August 2026

1. TASK OVERVIEW

The objective of Task 1 was to take the Olist Brazilian E-Commerce dataset provided as multiple CSV files and load it into a functioning local relational database.

The task consisted of three main stages:

1. Understanding the dataset, its tables, relationships, and intended machine-learning problem.
2. Downloading and ingesting the data into a relational database.
3. Connecting to the database and validating that the tables can be queried and joined successfully.

This report documents the practical work completed for these three stages and provides evidence that the database is ready for the subsequent stages of the MLOps training.

2. STEP 1 — READ & UNDERSTAND

2.1 Dataset Review

The dataset documentation and Kaggle dataset page were reviewed before beginning the database implementation.

The dataset is not provided as one machine-learning-ready table. Instead, information is distributed across related tables. The main relationships are centered around identifiers such as:

- order_id
- customer_id
- product_id
- seller_id

The Qafza documentation highlights that the eventual ML dataset will generally require aggregation before joining certain tables, particularly order items and payments, because these tables can contain multiple records for a single order.

This was an important consideration when reviewing the database structure, since simply joining every table at its original granularity could result in duplicated information.

2.2 Understanding the ML Problem

The recommended initial supervised-learning problem is late-delivery classification.

The intended prediction is whether a delivered order arrived after its estimated delivery date. The Qafza documentation identifies this as a suitable first supervised problem because it provides a clear binary target while requiring information from several related tables.

At this stage, no machine-learning model or feature engineering was performed. The purpose of this task was to establish the database foundation required for those later stages.

2.3 Dataset Relationships

The relationships between the main entities were inspected and represented through the database schema.

The implemented foreign-key relationships include:

- orders.customer_id → customers.customer_id
- order_items.order_id → orders.order_id
- order_items.product_id → products.product_id
- order_items.seller_id → sellers.seller_id
- order_payments.order_id → orders.order_id
- order_reviews.order_id → orders.order_id

The resulting ERD was generated from the PostgreSQL database using pgAdmin.

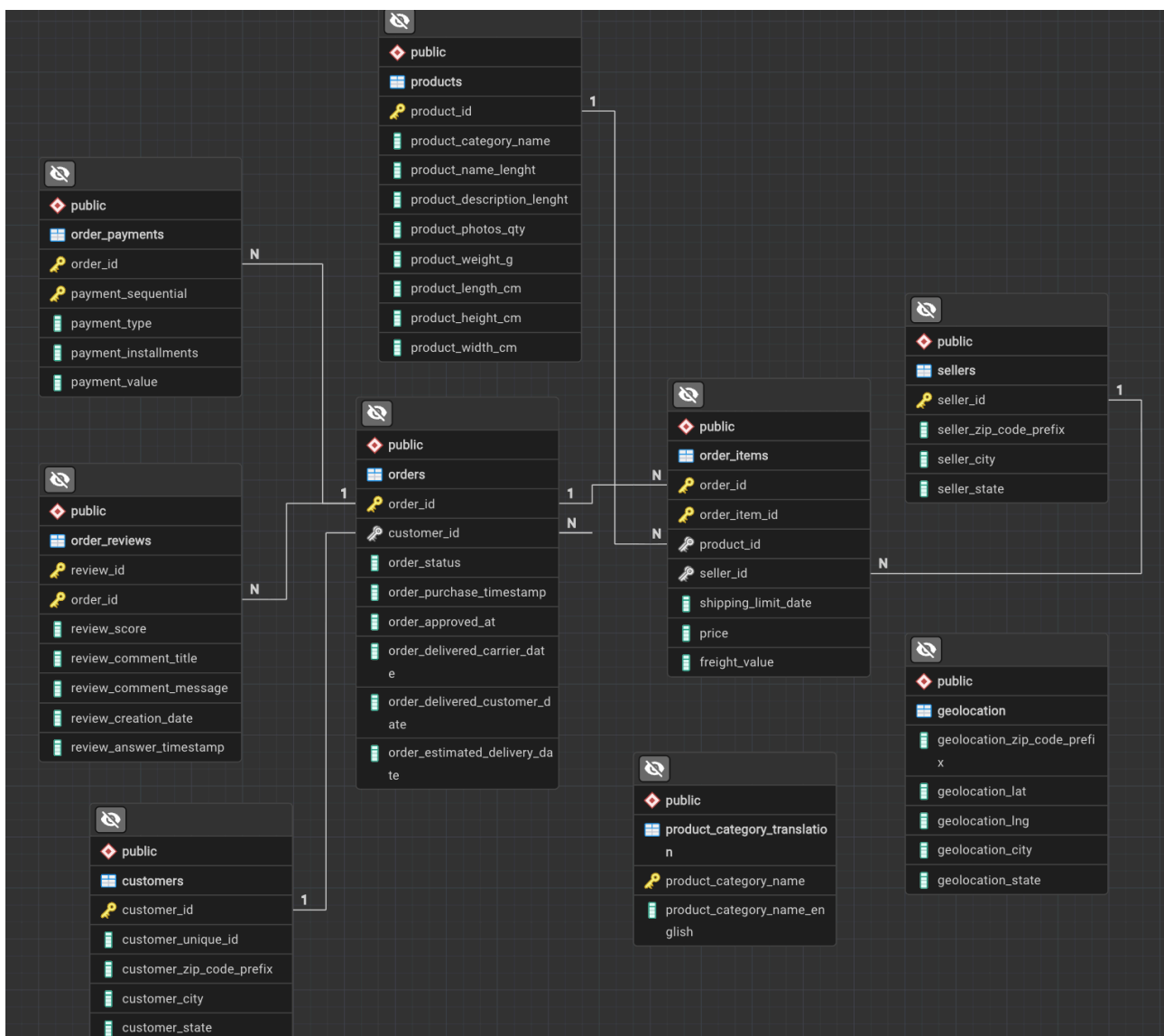


Figure 1 — Olist Database ERD

3. STEP 2 — DOWNLOAD & INGEST

3.1 Database Environment

The dataset was loaded into a local PostgreSQL installation and accessed through pgAdmin 4.

Database configuration:

Database system: PostgreSQL

Database name: olist

Host: localhost

Port: 5432

Database client: pgAdmin 4

The PostgreSQL server was successfully started locally and the olist database was created and populated.

3.2 Dataset Ingestion

The CSV files downloaded from the Olist dataset were loaded into the PostgreSQL database.

The database contains the following tables:

1. customers
2. orders
3. order_items
4. order_payments
5. order_reviews
6. products
7. sellers
8. geolocation
9. product_category_translation

The table structures were inspected after ingestion to confirm that the expected columns and data types were present.

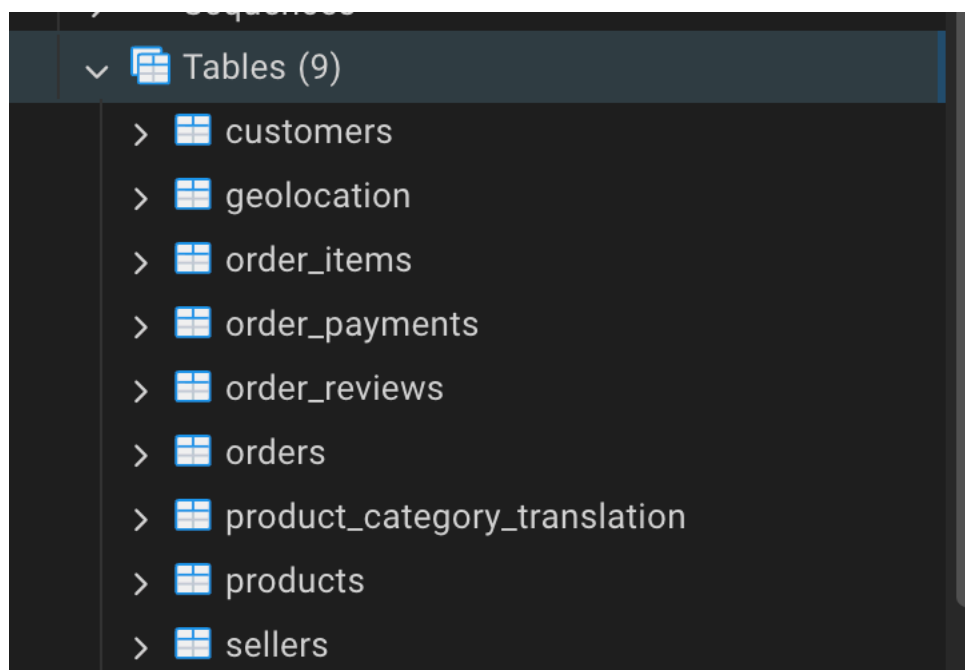


Figure 2 — Tables in the olist PostgreSQL Database

3.3 Database Constraints

Primary keys and foreign keys were inspected after the tables were created.

Primary keys were established for the main entity identifiers and composite keys where appropriate.

The foreign-key constraints were also verified through PostgreSQL database metadata.

This ensures that the relational structure is represented and enforced by the database where appropriate.

3.4 Data Ingestion Issue

During the initial import of the order_reviews table, PostgreSQL reported a duplicate primary-key violation:

ERROR: duplicate key value violates unique constraint "order_reviews_pkey"

DETAIL: Key (review_id)=(3242cc306a9218d0377831e175d62fbf) already exists.

The error occurred while importing the CSV because a review_id already existed in the table.

This was treated as a database integrity issue rather than bypassing the primary-key constraint.

The data in the database was subsequently checked and the table was brought into a state where the database could be queried successfully.

This incident also demonstrated the importance of database constraints during ingestion: the primary key prevented duplicate records from silently being inserted.

4. STEP 3 — TEST & VALIDATE

After ingestion, the database was tested through PostgreSQL queries executed using pgAdmin.

The validation queries are stored in:

tasks/task-01-database/sql/02_validation_queries.sql

The queries cover the following validation areas.

4.1 Table Verification

The database metadata was queried to confirm that the expected tables existed in the public schema.

The query uses PostgreSQL's information_schema.tables to retrieve the available tables.

This confirmed that the expected Olist tables were available for querying.

4.2 Row Count Verification

A row-count query was executed for each dataset table.

This was used to verify that data had actually been loaded into the database rather than only creating empty table structures.

The validation query covers:

- customers
- orders
- order_items
- order_payments
- order_reviews
- products
- sellers
- geolocation
- product_category_translation

4.3 Sample Data Query

Sample records were retrieved from the customers table to verify that individual records could be successfully read from PostgreSQL.

The successful result confirmed that the database was accessible and that the imported records could be queried.

4.4 Customer-Order JOIN

A JOIN between orders and customers was executed using their shared customer_id.

This test verified that information from two related tables could be combined through their relational key.

The screenshot shows a PostgreSQL client window with the following components:

- Query Editor:** Contains a SQL query that joins the 'orders' and 'customers' tables on 'customer_id' and limits the results to 10 rows.
- Data Output:** Displays the results of the query in a table format with 6 columns: order_id, order_status, order_purchase_timestamp, customer_city, and customer_state.
- Footer:** Shows 'Total rows: 10' and 'Query complete 00:00:00.145'.

Query:

```

1 SELECT
2     o.order_id,
3     o.order_status,
4     o.order_purchase_timestamp,
5     c.customer_city,
6     c.customer_state
7 FROM orders o
8 JOIN customers c
9     ON o.customer_id = c.customer_id
10 LIMIT 10;

```

Data Output:

	order_id character varying (50)	order_status character varying (30)	order_purchase_timestamp timestamp without time zone	customer_city character varying (100)	customer_state character varying (2)
1	e481f51cbdc54678b7cc49136f2d6a...	delivered	2017-10-02 10:56:33	sao paulo	SP
2	53cdb2fc8bc7dce0b6741e2150273...	delivered	2018-07-24 20:41:37	barreiras	BA
3	47770eb9100c2d0c44946d9cf07ec...	delivered	2018-08-08 08:38:49	vianopolis	GO
4	949d5b44dbf5de918fe9c16f97b45f...	delivered	2017-11-18 19:28:06	sao goncalo do amaran...	RN
5	ad21c59c0840e6cb83a9ceb5573f8...	delivered	2018-02-13 21:18:39	santo andre	SP
6	a4591c265e18cb1dcee52889e2d8a...	delivered	2017-07-09 21:57:05	congonhinhos	PR
7	136cce7faa42fdb2cefd53fdc79a6098	invoiced	2017-04-11 12:22:08	santa rosa	RS
8	6514b8ad8028c9f2cc2374ded2457...	delivered	2017-05-16 13:10:30	nilopolis	RJ
9	76c6e866289321a7c93b82b54852d...	delivered	2017-01-23 18:29:09	faxinalzinho	RS
10	e69bfb5eb88e0ed6a785585b27e16...	delivered	2017-07-29 11:55:02	sorocaba	SP

Total rows: 10 Query complete 00:00:00.145 LF Ln 1, Col 7

Figure 3 — Customer and Order JOIN Result

4.5 Order–Item–Product JOIN

A second JOIN was performed across three tables:

```
orders
  ↓
order_items
  ↓
products
```

The query connected:

- orders.order_id
- order_items.order_id
- order_items.product_id
- products.product_id

This verified that the database could combine order-level and product-level information through the implemented relationships.

4.6 Order–Product–Seller JOIN

A multi-table JOIN was also tested between orders, order items, products, and sellers.

The query retrieves information including:

- order ID
- product category
- seller city
- seller state
- item price

This demonstrated that the database supports queries spanning multiple related entities.

4.7 Delivery Information Validation

Because the eventual ML problem concerns delivery performance, the order table was also queried for the relevant delivery timestamps.

This confirms that the database contains the timestamps required to construct the eventual delivery outcome.

At a later stage, these timestamps can be used to determine whether a delivered order arrived after its estimated delivery date.

5. ADDITIONAL DATABASE OBSERVATIONS

During the database validation process, two aspects of the dataset structure were specifically observed.

5.1 Geolocation

The geolocation table contains multiple records for the same ZIP-code prefix. Therefore, geolocation_zip_code_prefix is not a unique identifier that can directly function as a primary key.

For example, querying the table by ZIP-code prefix showed multiple occurrences for the same prefix.

This means the relationship between geographical information and customer/seller ZIP-code prefixes is a logical data relationship rather than a simple one-to-one foreign-key relationship.

5.2 Product Category Translation

The `product_category_translation` table provides English translations for Portuguese product category names.

The database was queried to inspect product categories without translations. The result included categories such as:

- `portateis_cozinha_e_preparadores_de_alimentos`
- `pc_gamer`

This confirms that the translation table should not be assumed to contain a translation for every category appearing in the products table.

6. EVIDENCE OF COMPLETION

The following evidence was produced during the task.

Database:

- Local PostgreSQL server successfully running.
- olist database created.
- Nine dataset tables populated.
- Primary and foreign-key constraints verified.

SQL:

The repository contains:

`tasks/task-01-database/sql/01_create_tables.sql`
`tasks/task-01-database/sql/02_validation_queries.sql`

The first file contains the database table definitions and constraints.

The second contains some of the validation queries used to test the database.

ERD:

An ERD generated from the PostgreSQL database is included in:

`docs/erd/olist_erd.png`

Screenshots:

pgAdmin screenshots provide visual evidence of successful queries and JOIN results.

7. TASK COMPLETION CHECKLIST

- Review and understand the dataset ✓
- Understand the main table relationships ✓
- Understand the late-delivery prediction problem ✓
- Download the dataset ✓
- Load the data into a relational database ✓
- Database running locally ✓
- Query the tables ✓
- Test JOIN operations ✓
- Verify database structure ✓

8. CONCLUSION

Task 1 — Get the Data Into a Database — has been completed.

The Olist dataset was downloaded from its original CSV format and loaded into a local PostgreSQL relational database. The resulting database contains the required dataset tables, with primary and foreign-key relationships established where appropriate.

The database was subsequently validated through metadata queries, row-count checks, sample data retrieval, and JOIN operations across related tables. Delivery-related timestamps were also queried to confirm that the database contains the information required for the subsequent late-delivery classification task.

The work completed in this task establishes the database foundation for the next stages of the MLOps project, where the data can be explored, cleaned, transformed, and eventually prepared for machine-learning purposes.